

URBAN EYE

Autonomous Smart-City Infrastructure Surveillance, Computer Vision Defect Recognition, Geospatial Deduplication & Instant Municipal Work Order Dispatch

Executive Summary: Urban EYE is an end-to-end, production-grade smart-city surveillance platform engineered to replace slow, manual citizen complaint triage with automated defect recognition, geospatial clustering, and instantaneous municipal work order dispatch. When citizens capture photos or video feeds of municipal hazards (potholes, garbage dumping, water pipe bursts, broken streetlights), Urban EYE's multi-stage pipeline executes client-side compression, binary EXIF GPS geotagging, YOLOv8 deep-learning computer vision inference, rule-based severity classification, 50-meter/30-day duplicate clustering, municipal department routing, and automated email dispatch within 2 to 3 seconds. The platform bridges the gap between active citizen participation and agile civic administration.

Document Scope:	Full Architecture, End-to-End Technology Stack, Pipelines, Schemas & API Contracts
System Edition:	Urban EYE v1.0.0 Production Release
Architectural Tiers:	Python FastAPI AI Microservice Node.js Express API & Socket.io React 18 UI MongoDB
Key Breakthroughs:	Sub-3s Edge-to-Dispatch Triage, 50m/30d Geospatial Deduplication, Closed-Loop Citizen Notification
Verification Mode:	Verifiable Nodemailer Ethereum Preview Links + Live Socket.io Progress Stepper
Report Date:	September 2026

TABLE OF CONTENTS	
1. The Core Idea, Problem Statement & Civic Impact	5. Downstream Severity & Urgency Classifier
2. System Architecture & Component Design	6. Geospatial Clustering & 2dsphere Database
3. Complete Technology Stack (Without Omission)	7. REST API & WebSocket Event Specifications
4. The 6-Stage Autonomous Incident Pipeline	8. User Portals, Security & Deployment Blueprint

1. The Core Idea, Problem Statement & Civic Impact

1.1 The Municipal Infrastructure Dilemma

Civil infrastructure is the lifeline of modern urban civilization. However, public assets undergo constant decay due to weather, heavy traffic, and civic neglect. In traditional municipal workflows, city departments operate reactively, relying on citizen complaint hotlines or web portals. This legacy approach suffers from critical structural flaws:

- **High Redressal Latency:** Grievances sit unread in general municipal inboxes for days before human clerks categorize and route them.
- **Duplicate Report Proliferation:** A single pothole on a major arterial road often generates 20 to 50 separate citizen complaints. Each creates a disjointed ticket, inflating backlog metrics and misallocating inspection crews.
- **Subjective and Unverified Severity:** Without objective computer vision quantification, complaints cannot be accurately prioritized.
- **Bureaucratic Black Hole:** Citizens rarely know if their report was dispatched or repaired, leading to civic frustration and voter apathy.

1.2 The Urban EYE Vision: Autonomous Civic Governance

Urban EYE transforms this manual, error-prone cycle into a self-driving civic dispatch pipeline. Any visual submission (mobile photo, live camera stream, or uploaded media) is instantaneously analyzed by an AI computer vision microservice, matched against existing geo-clusters within 50 meters, assigned a mathematical severity score, packaged into a formal legal complaint letter, and delivered directly to the designated department's mailbox with verifiable proof.

1.3 Targeted Civil Hazard Categories

Category	Hazard & Risk Profile	Assigned Municipal Authority	Mandated Action Plan
Potholes	Carriageway craters, asphalt fissures, and road surface collapses causing two-wheeler fatalities, vehicle suspension damage, and acute traffic bottlenecks.	Roads & Public Works Dept (PWD)	Dispatch road crew with hot/cold-mix asphalt patch unit to level surface.
Garbage Dumping	Illegal solid waste dumps, uncollected community refuse heaps generating noxious leachate, pest infestation, and public bio-sanitary hazards.	Public Health & Sanitation Dept	Deploy compactor trucks, sanitation sweepers, and disinfectant powder.
Water Leakages	Ruptured distribution mains and hydrant failures wasting potable municipal water, eroding sub-road foundations, and causing localized inundation.	Water Supply & Sewerage Dept	Isolate pipe sector valve, excavate subterranean conduit, replace pipe segment.
Faulty Streetlights	Defective luminaires and severed underground cables plunging public roads into darkness, facilitating pedestrian danger and night collisions.	Electrical & Public Lighting Dept	Depute bucket-truck unit to replace LED luminaire/choke and repair wiring.

2. System Architecture & High-Level Design

Urban EYE implements a decoupled microservices pattern across four discrete layers: Presentation (React 18), API Gateway & Real-Time Hub (Node.js/Express), Computer Vision Microservice (Python FastAPI/YOLOv8), and Persistence & Spatial Indexing (MongoDB/Mongoose).

```
+-----+ | PRESENTATION
TIER (CLIENT) | | React 18 + Vite | Leaflet / Google Maps GIS | Recharts Analytics | Socket.io Client |
+-----+ | Photo Upload
(Compressed JPEG + Multipart Form Data) ^ Real-Time Progress v | (Socket.io Rooms)
+-----+ | API GATEWAY &
BUSINESS ENGINE (NODE.JS) | | Express 4.19 | Multer MemoryStorage | JWT RBAC | Rate Limiting | Socket.io Hub |
+-----+ | | | | POST
/detect Form-Data | 2dsphere Spatial Query | Media Storage | SMTP v v v v +-----+
+-----+ +-----+ +-----+ | AI FASTAPI CORE | | MONGODB ATLAS | |
CLOUDINARY/LOCAL | | NODemailer | | Python 3.10+ | | 2dsphere Index | | Storage Adapter | | Ethereum/ | |
Ultralytics | | Dual In-Memory | | Evidence Photos | | Resend/ | | YOLOv8 PyTorch | | Fallback | | CDN / Disk
| | Gmail SMTP | +-----+ +-----+ +-----+ +-----+ | | |
+----- [BBox, Conf, Type] --+ [50m Deduplication & Escalation] v [MUNICIPAL DEPT]
```

2.1 Microservice Responsibilities & Inter-Service Communication

Subsystem	Runtime Stack	Boundary, Responsibilities & Protocols
AI Inference Microservice	Python 3.10+ FastAPI / Uvicorn Ultralytics YOLOv8	Dedicated computational node. Accepts raw binary image buffers via POST /detect; executes YOLOv8 model prediction; outputs strict Pydantic bounding boxes { x, y, width, height }, class labels, and confidence scores. Exposes GET /health.
API Gateway & Triage Core	Node.js 18+ / 24+ Express.js 4.19 Socket.io 4.7	Central controller. Handles user auth, JWT validation, client-side EXIF GPS injection, image persistence, inter-service proxying to AI, spatial duplicate clustering, formal complaint generation, email dispatch, and WebSockets broadcasting.
Spatial Persistence Layer	MongoDB 7.0 Mongoose 9.x MongoMemoryServer	Maintains Detection, User, and Announcement collections. Utilizes GeoJSON Point with 2dsphere indexing for sub-millisecond proximity queries. Employs automatic in-memory failover if cloud network/IP whitelisting fails.
Command Center & Client UI	React 18.2 Vite 5.1 Tailwind CSS 3.4 Leaflet GIS	Delivers rich citizen reporting forms with live camera, client compression, 6-stage animated stepper, GIS map with pulsating pins, and administrative triage dashboard.

3. Complete Technology Stack (Without Omission)

Every single library, engine, protocol, and driver deployed across the Urban EYE platform is specified below:

3.1 Computer Vision & AI Microservice (Python Tier)

Technology / Library	Version	Specific Purpose & Architectural Role
Python	>= 3.10	High-performance core runtime environment for numerical computation and tensor processing.
FastAPI	>= 0.110.0	Asynchronous web framework exposing high-throughput REST inference endpoints with automatic schema validation.
Uvicorn [standard]	>= 0.28.0	High-performance ASGI server with uvloop and httptools handling concurrent network requests.
Ultralytics	>= 8.1.0	YOLOv8 framework executing computer vision inference, non-maximum suppression (NMS), and box extraction.
PyTorch (torch, torchvision)	>= 2.0.0	Deep learning tensor computation engine powering neural network weight loading and CUDA GPU acceleration.
Pillow (PIL)	>= 10.2.0	Image decoding, format normalization (RGB conversion), and in-memory byte buffer manipulation.
Pydantic	>= 2.6.0	Data contract enforcement, verifying strict request parameters and BoundingBox/Detection response models.
Python-Multipart	>= 0.0.9	Streaming multipart/form-data parser for processing raw uploaded image files from Node.js.
Python-Dotenv	>= 1.0.0	Parses .env configuration controlling the YOLOv8 model path and stub detector toggle.

3.2 Backend API, Real-Time & Triage Engine (Node.js Tier)

Technology / Library	Version	Specific Purpose & Architectural Role
Node.js	v18+ / v24+	Event-driven JavaScript runtime executing the API Gateway and asynchronous business logic.
Express.js	^4.19.2	HTTP web framework handling routing, middleware chaining, and RESTful service endpoints.
Socket.io	^4.7.5	WebSocket engine providing two-way real-time event streaming for the 6-stage pipeline and dashboard synchronization.
Mongoose	^9.9.4	Object Document Mapper providing schema enforcement, lifecycle hooks, and 2dsphere index management.
mongodb-memory-server	^11.2.0	Zero-config local in-memory MongoDB instance providing 100% automated fallback if cloud Atlas is blocked.
Nodemailer	^6.9.13	Transactional email client managing automated dispatch, custom SMTP, Gmail, and Ethereum test accounts.
Resend REST API	REST v1	Direct HTTP email dispatch fallback that bypasses cloud host (e.g. Render) port 25/587 outbound SMTP blocks.
Multer	^1.4.5-lts.1	MemoryStorage file upload middleware buffering image streams up to 20MB before processing.
Cloudinary SDK	^2.11.0	Cloud media storage provider managing evidence photos with secure CDN delivery and local disk fallback.
Exif-Parser	^0.1.12	Binary metadata decoder extracting GPS latitude/longitude directly from incoming JPEG buffers.
Piexifjs	^1.0.6	Binary EXIF injector converting manual GPS coordinates into standard EXIF Rational format inside JPEG buffers.
Axios	^1.6.8	Promise-based HTTP client dispatching form-data payloads to the Python AI service and geocoding queries.
Form-Data	^4.0.0	Constructs multi-part HTTP request streams for passing image buffers across process boundaries.
JSONWebToken (JWT)	^9.0.2	Stateless cryptographic bearer token generation and verification for Role-Based Access Control.
Bcryptjs	^2.4.3	One-way cryptographic password hashing using 10 salt rounds for secure credential storage.
Express-Rate-Limit	^7.2.0	Sliding-window IP rate limiting mitigating burst upload spam and brute-force authentication attacks.
UUID	^14.0.2	RFC4122 compliant UUIDv4 generator for incident references, job tracking IDs, and file naming.
Dotenv	^16.6.1	Loads environment variables from .env files into process.env across development and production.
CORS	^2.8.5	Configures Cross-Origin headers allowing web clients to communicate securely with the API.

3.3 Presentation & Command Center (React Tier)

Technology / Library	Version	Specific Purpose & Architectural Role
React	^18.2.0	Component architecture managing declarative UI rendering, concurrent features, and state.
Vite	^5.1.6	Modern build tool providing native ES module bundling and fast Hot Module Replacement.
React Router DOM	^6.22.3	Client-side routing engine managing authenticated route guards and role-based redirects.
Socket.io-client	^4.7.5	Client WebSocket subscriber listening to 6-stage pipeline progress and global incident broadcasts.
Leaflet & OpenStreetMap	1.9.4	Interactive GIS map view with OpenStreetMap tile layers, pulsating severity markers, and cluster boundaries.
@vis.gl/react-google-maps	^1.9.0	Official Google Maps React wrapper for satellite hybrid imagery and high-accuracy street geocoding.
Recharts	^3.10.1	Composable SVG charting library rendering incident trend graphs, department distributions, and resolution velocity.
Browser-Image-Compression	^2.0.2	Client-side image optimizer downsampling high-res smartphone photos (1600px max, 80% quality) prior to transit.
Exifr	^7.1.3	Client-side EXIF extractor reading GPS tags directly from file inputs before network transmission.
Lucide React	^0.359.0	Comprehensive icon system providing visual cues for issue categories, severity ranks, and navigation.
jsPDF	^4.2.1	Client-side vector PDF generator allowing citizens and admins to export official grievance letters.
Tailwind CSS	^3.4.1	Utility-first CSS framework enabling responsive layout, custom themes, dark-mode styling, and micro-interactions.
PostCSS & Autoprefixer	^8.4.35	CSS compilation engine handling automated vendor prefixing and modern CSS parsing.
Tailwind-Merge & Clsx	^2.2.2	Dynamic utility class conditional concatenation without CSS rule conflicts.

3.4 Infrastructure, Orchestration & DevOps Tier

Technology / Tool	Specification	Operational Deployment & Infrastructure Role
Docker	Engine v24+	Container engine encapsulating Node.js, Python, MongoDB, and frontend runtimes into isolated images.
Docker Compose	v3.8 Compose Spec	Multi-container orchestration orchestrating 4 services (mongodb, ai_service, backend, frontend).
Docker Named Volumes	mongodb_data	Persistent host storage ensuring 100% database survival across container rebuilds.
Bridge Networking	urban_eye_network	Isolated virtual bridge network enabling internal service discovery and secure container routing.
OpenStreetMap Nominatim	REST API	Reverse geocoding engine translating decimal lat/ing coordinates to standard municipal street addresses.
Ethereal Email	Virtual SMTP	Provides zero-configuration verifiable email inbox URLs for testing automated work orders.

4. The 6-Stage Autonomous Incident Pipeline

When an image is submitted via POST /api/detect, Urban EYE executes a synchronized 6-stage pipeline. Each stage boundary emits a pipeline:progress WebSocket event to the submitting client's UI room, enabling real-time animated stepper visualization.

Stage 1: Received & Ingested	Payload Validation & Geotag Injection: The uploaded image is received into memory via Multer. The system extracts GPS coordinates from image EXIF metadata (via exif-parser). If missing, it uses frontend pin coordinates. The system then uses piexifjs to inject binary GPS EXIF tags directly into the JPEG buffer before persisting the file to Cloudinary or local disk.
Stage 2: AI Computer Vision	YOLOv8 Deep Learning Inference: The image buffer is dispatched to the Python FastAPI microservice. The model predicts the defect type (pothole, garbage, water_leak, streetlight), calculates detection confidence (0.0 to 1.0), and extracts precise bounding box dimensions (x, y, width, height). An integration seam allows instant zero-code swapping between trained weights and a deterministic stub.
Stage 3: Geocoding & Address	Reverse Geocoding Standardization: GPS decimal coordinates are resolved into human-readable physical addresses using Google Maps Geocoding API with automated fallback to OpenStreetMap Nominatim and localized ward naming heuristics.
Stage 4: Geospatial Clustering	50-Meter / 30-Day Deduplication: MongoDB executes a 2dsphere proximity search (\$near, 50m radius, non-resolved status within past 30 days). If a match is found, the report is merged, reportCount increments, the user ID is added to reporterIds, and severity is re-evaluated. If no match exists, a new incident is created.
Stage 5: Department Routing	Authority Mapping & Grievance Generation: The defect type is mapped against departments.json to determine the assigned authority. routingService.js generates a formal legal complaint letter including Complaint Reference No, hazard description, citizen impact, and legal relief prayer.
Stage 6: Instant Municipal Dispatch	Automated Work Order Email & Broadcast: The work order is dispatched via Nodemailer/SMTP (or Resend API) to the department's inbox. An Ethereum preview link is generated for administrative verification. Status transitions to ASSIGNED and a detection:created event is broadcast across all command centers.

5. Rule-Based Severity & Urgency Classifier

To prevent false alarms and reflect real-world community impact, Urban EYE combines AI confidence with physical defect geometry and citizen report volume into an objective composite score:

Composite Severity Score Formula:

Composite Score = (S_confidence + S_area) * W_type + S_community

- **S_confidence (Max 35 pts):** $\geq 0.85 \rightarrow 35$ pts | $\geq 0.65 \rightarrow 25$ pts | $< 0.65 \rightarrow 15$ pts
- **S_area (Max 35 pts):** $\geq 70,000 \text{ px}^2 \rightarrow 35$ pts | $\geq 25,000 \text{ px}^2 \rightarrow 25$ pts | $< 25,000 \text{ px}^2 \rightarrow 15$ pts
- **W_type (Hazard Multiplier):** Water Leak = 1.25x | Pothole = 1.15x | Streetlight = 1.10x | Garbage = 1.00x
- **S_community (Surge Escalation):** 2+ Citizen Reports $\rightarrow +20$ pts | 4+ Citizen Reports $\rightarrow +35$ pts

Priority Thresholds: HIGH Priority: Score ≥ 75 or Reports ≥ 4 | MEDIUM Priority: Score ≥ 45 or Reports ≥ 2 | LOW Priority: Score < 45

Automatic Surge Escalation: When duplicate reports push an incident's severity rank up (e.g. from Low to Medium, or Medium to High), the system automatically triggers an **Escalation Alert Email** to the department head and emits a WebSocket event causing the incident pin on the command center GIS map to pulse vigorously.

6. Geospatial Clustering & Database Architecture

Urban EYE uses MongoDB's **2dsphere geospatial index** to evaluate physical distances over Earth's spherical surface. The schema guarantees high performance across spatial queries, reporter tracking, and resolution notifications.

Field Name	BSON Type	Indexing / Constraints	Functional Purpose in Urban EYE
id	String	Unique (UUIDv4)	Deterministic external incident reference (e.g. UE-A8F3C192).
location	GeoJSON Object	2dsphere Spatial Index	Stores { type: 'Point', coordinates: [lng, lat] } for \$near proximity clustering.
lat, lng	Number (Double)	Required	Decimal latitude and longitude coordinates extracted from EXIF or map pin.
type	String	Indexed (Enum)	Defect category: pothole garbage water_leak streetlight.
confidence	Number	0.0 to 1.0	Deep learning detection confidence output by YOLOv8 model.
severity	String	Indexed (low/med/high)	Dynamic severity calculated via multi-factor rule engine.
status	String	new assigned resolved	Municipal lifecycle state. Setting to resolved triggers citizen notices.
reportCount	Number	Default 1	Count of independent citizen reports merged into this specific spatial cluster.
reporterIds	Array of Strings	Array of User IDs	Audit list of all citizens who contributed reports to this incident cluster.
reportText	String	Full Text	Formal legal civic complaint letter generated for municipal dispatch.
dispatchStatus	String	pending sent failed	Delivery tracking flag for the outbound email work order.
dispatchPreviewUrl	String	URL string	Verifiable Ethereum test inbox link allowing live review of the dispatched email.

6.1 Dual Persistence & Zero-Downtime Fallback Architecture

To guarantee 100% operational resilience during demonstrations, evaluation, or offline deployments, backend/config/database.js implements an automated failover hierarchy: it first attempts connection to MongoDB Atlas (timeout: 5000ms). If blocked by cloud network policies or firewall restrictions, it seamlessly launches an embedded in-memory MongoDB daemon (MongoMemoryServer) without interrupting application availability.

7. REST API & WebSocket Event Specifications

7.1 RESTful API Endpoint Directory

Method	Route	Auth Scope	Payload & Functional Description
POST	/api/auth/register	Public	Creates a citizen account with name, email, phone, and bcrypt password hash.
POST	/api/auth/login	Public	Authenticates credentials; issues signed JWT bearer token with user role.
GET	/api/auth/me	JWT Bearer	Returns current authenticated user profile, permissions, and neighborhood.
POST	/api/detect	JWT Bearer	Primary triage trigger. Multipart upload (image, lat, lng). Executes full 6-stage pipeline.
GET	/api/detections	JWT Bearer	Fetches detections list. Admins access citywide ledger; citizens receive own reports.
GET	/api/detections/:id	JWT Bearer	Returns single incident record with evidence image, bounding boxes, and work order.
PATCH	/api/detections/:id/status	Admin Only	Updates status (new, assigned, resolved). 'resolved' triggers citizen resolution notices.
PATCH	/api/detections/:id	JWT Bearer	Edits formal grievance text, corrected physical address, or citizen observations.
DELETE	/api/detections/:id	Admin / Owner	Removes incident record; broadcasts deletion event to GIS map across all clients.
GET	/api/stats/summary	Admin Only	Calculates system totals, status counts, breakdown by type, and department workload.
GET	/api/stats/public-summary	Public	Returns resolved count and active citizen metrics for the public landing page.
GET	/api/stats/user-summary	JWT Bearer	Calculates citizen personal contributions, resolved count, and civic trust score.
GET	/api/announcements	JWT Bearer	Retrieves active municipal emergency broadcasts and civic maintenance notices.
POST	/api/announcements	Admin Only	Publishes citywide public advisory with urgency level (normal, urgent, emergency).

7.2 Real-Time WebSocket Event Contracts (Socket.io)

Event Identifier	Transmission Scope	Payload Schema & Operational Significance
pipeline:progress	Job Room & Broadcast	{ jobId, stage, step, totalSteps: 6, message, details } Fired at each of the 6 pipeline stage boundaries to animate the frontend stepper in real time.
detection:created	Global Broadcast	{ ...detectionRecord } Notifies all active command centers of a brand-new, non-duplicate municipal issue dispatch.
detection:merged	Broadcast & User	{ detection, details: { isDuplicate, reportCount, escalated } } Informs clients that an existing incident absorbed a duplicate report. Updates map pin pulse if escalated.
detection:updated	Broadcast & Reporters	{ ...updatedDetectionRecord } Broadcasts administrative triage changes (e.g. status changed to resolved). Citizens receive immediate resolution banner.
detection:deleted	Broadcast & Reporters	{ id: detectionId } Removes pin from GIS maps and incident tables instantly across all connected screens.
announcement:created	Global Broadcast	{ ...announcementRecord } Pushes critical municipal emergency alerts and maintenance schedules to all active users.

8. User Portals, Security & Deployment Blueprint

8.1 Citizen Portal Experience

Designed for rapid field reporting from mobile devices with zero friction:

- **Direct Camera Capture:** Uses HTML5 media stream APIs for instant rear camera capture or desktop webcam snapshots.
- **Client-Side Image Optimization:** Compresses multi-megabyte photos to 1600px max at 80% JPEG quality before network transit.
- **Real-Time 6-Stage Progress Stepper:** Connects to Socket.io to visually display progress through Received, Detecting, Geotagging, Deduplication, Routing, and Dispatched.
- **Closed-Loop Resolution Feedback:** Citizens receive automated resolution emails the moment field crews resolve their reported issue.
- **Civic Trust Scoring:** Citizens earn trust score increments upon verified resolutions, gamifying community civic responsibility.

8.2 Municipal Command Center (Admin Experience)

Provides municipal decision-makers with comprehensive situational awareness:

- **Interactive GIS Heatmap:** Live geospatial map plotting all active city incidents with pulsating severity markers (Red = High, Orange = Medium, Green = Low).
- **Incident Triage Ledger:** Real-time table with instant multi-facet filtering (defect type, severity, status, assigned department) and search.
- **Verifiable Email Previewer:** Direct modal button to open the actual Ethereum work order dispatch email for audit and inspection.
- **Executive Workload Analytics:** Dynamic Recharts breakdowns of department volume, average turnaround time, and resolution velocity.

8.3 Security & Hardening Architecture

- **Role-Based Access Control (RBAC):** JWT tokens encode user roles (`user` vs `admin`). Administrative routes are protected by requireAdmin middleware.
- **Cryptographic Hashing:** Passwords hashed using bcrypt with 10 salt rounds. Strict complexity rules enforced.
- **Sliding-Window Rate Limiting:** express-rate-limit prevents upload spamming and brute-force authentication attacks.
- **Sanitized Media Ingestion:** Image buffers are validated against magic bytes, stripped of potential code payloads, and capped at 20MB.

8.4 Docker Multi-Container Orchestration

Container Architecture & Ports:

- **urban_eye_db (mongo:7.0):** Port 27017 | Volume: mongodb_data:/data/db
 - **urban_eye_ai (Python FastAPI):** Port 8000 | Ultralytics YOLOv8 inference service
 - **urban_eye_api (Node.js/Express):** Port 5000 | Volumes: uploads, data | Depends on Mongo & AI
 - **urban_eye_ui (React/Vite):** Port 5173 | Command Center Web Application | Connects to Backend API
- Single-Command Launch:** docker-compose up --build

Conclusion: Urban EYE establishes a new paradigm in smart-city civil engineering. By replacing manual complaint backlogs with autonomous computer vision, intelligent 50m spatial deduplication, rule-based severity escalation, and instant verifiable municipal dispatch, the platform delivers the speed, accountability, and operational intelligence demanded by modern 21st-century cities.